

KOBIE LAUNCHPAD AI HACKATHON 2026

Phase 1 Solution Proposal

One - Click Environment Provisioner

Track: Cloud Engineering

Team Name: **One-Click-Ops**

College Name: **PES UNIVERSITY**

Team Member 1: **Vaishnavi J P [PES1UG23CS900]**

Email-Id: vaishjp2005@gmail.com

Team Member 2: **Vivek G**

[PES1UG23CS703]

Email-Id: bhatvivek955@gmail.com

Department: **Computer Science Engineering**

Summary: We are building a system that transforms a single command into a fully operational cloud environment with networking, database, Kubernetes infrastructure, platform tooling, and a deployed three-tier application without manual intervention. Our One-Click Environment Provisioner uses modular Terraform to provision a production-grade AWS stack with VPC, RDS, and EKS, alongside secure remote state management and least-privilege IAM. FluxCD manages platform and application deployments declaratively through Git, preventing configuration drift. A Jenkins CI/CD pipeline handles testing, vulnerability scanning, and deployment with strict failure gates. The platform is idempotent, secure, observable through Prometheus and Grafana, and designed with realistic MVP scope and risk mitigation strategies.

1.Problem Understanding

Actual Challenge in building this application: The real challenge is building a system that is **idempotent**, **declarative**, and **GitOps-native**, meaning a single trigger must produce the same correct environment regardless of what state the infrastructure was previously in, and that every change flow through version control. This is fundamentally different from writing a Terraform script that works once.

7 main problems are stated below along with their description and sub modules that are being implemented:

- **The Bootstrap Ordering Problem:** EKS must be provisioned before FluxCD installation, and FluxCD must be operational before deploying platform services and the 3-tier application. Since Terraform creates infrastructure but cannot bootstrap GitOps directly, the transition requires an idempotent orchestration layer that bridges infrastructure provisioning, platform initialization, and application deployment in the correct sequence.
- **Terraform State Management & Drift:** Terraform state files track what Terraform believes exists in AWS. If actual infrastructure changes due to manual edits, failed deployments, or AWS-side events, future applies can fail or trigger unintended changes. This requires secure remote state management with locking, continuous drift detection, and strict controls preventing unauthorized manual modifications to cloud infrastructure.
- **Least-Privilege Secret Propagation:** EKS workloads require AWS access for services like RDS and ECR. Assigning permissions at the node level exposes every pod to the same AWS privileges, creating security risks. Using IRSA enables pod-level least-privilege access through correctly configured OIDC integration, scoped IAM roles, and Kubernetes ServiceAccount mappings deployed in a strict sequence.
- **GitOps Reconciliation (Not One-Time Deployment):** FluxCD must continuously synchronize Kubernetes cluster state with the Git repository, even after deployment failures or manual changes. Reliable reconciliation requires dependency-aware deployment ordering, continuous health validation, and automated alerting whenever synchronization is delayed, suspended, or unsuccessful, ensuring the platform remains consistent, self-healing, and operationally stable at all times.
- **Autoscaling Coordination Across Three Layers:** Autoscaling must function cohesively across Kubernetes pods, EC2 worker nodes, and database replicas to maintain application stability under varying workloads. If pod scaling occurs before additional nodes are provisioned, workloads remain stuck in a Pending state. Proper coordination between HPA, Cluster Autoscaler, and database scaling mechanisms is essential for reliable and uninterrupted platform operation.
- **Closing the CI/CD Loop:** A CI/CD pipeline is incomplete if Jenkins only builds and pushes Docker images. The workflow must also update the GitOps repository with the new image version and verify successful FluxCD reconciliation before marking deployment successful, ensuring continuous integration and continuous delivery remain fully synchronized, validated, and operationally connected throughout the deployment lifecycle.

- **Security & Secret Exposure Risks:** CI/CD systems, Kubernetes workloads, and cloud infrastructure handle sensitive credentials, image artifacts, and deployment permissions. Misconfigured IAM roles, exposed secrets, permissive policies, or unscanned images can compromise the platform. Mitigation requires least-privilege IAM, secure secrets, vulnerability scanning, encrypted state, and controlled access across all layers.

Given problem statement	Extended Analysis of given problem statement
Terraform for IaC with VPC, RDS, and EKS as core infrastructure	Modular Terraform architecture with separate networking, database, and EKS modules, using S3 remote state storage, DynamoDB locking, and KMS encryption
FluxCD + Helm for platform GitOps	FluxCD selected over ArgoCD for pull-based GitOps, native dependency ordering, and reduced cluster credential exposure
Jenkins CI/CD for 3-tier application	Jenkins deployed inside EKS in a dedicated namespace, handling infrastructure orchestration and application CI/CD pipelines
End-to-end automation and GitOps as design principle	Fully automated workflow integrating Terraform, Jenkins, FluxCD, Helm, Kubernetes, and GitOps reconciliation without manual kubectl operations
Autoscaling as a requirement	HPA for pod scaling, Cluster Autoscaler for EC2 node scaling, and coordinated scaling strategy to prevent pending workloads

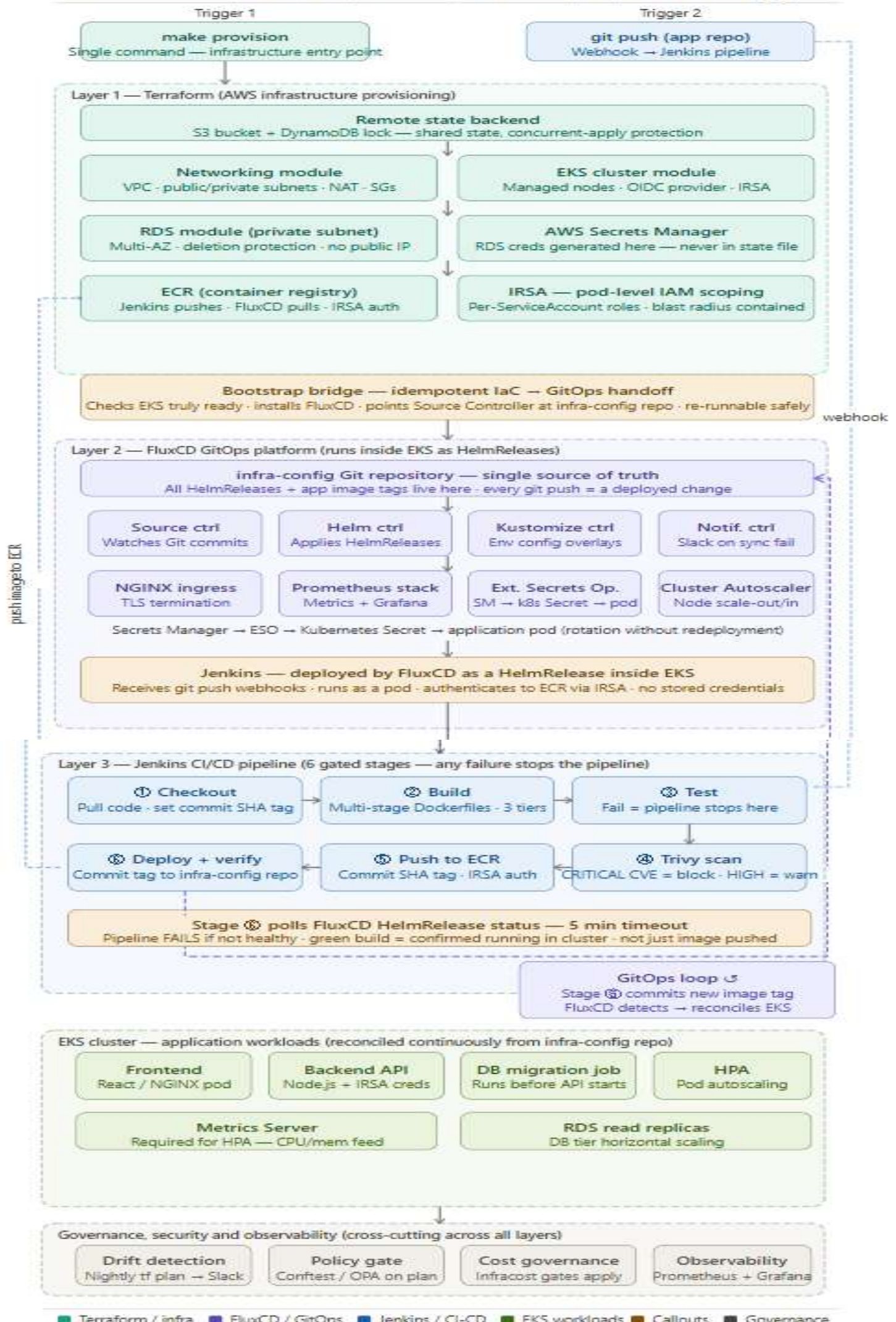
2. Proposed Approach & Solution Design

Architecture Overview: Our solution follows a three-layer architecture where Terraform, FluxCD, and Jenkins each handle distinct responsibilities and can be independently tested or replaced. Terraform provisions declarative, idempotent infrastructure but lacks continuous reconciliation, which FluxCD solves through GitOps synchronization. FluxCD maintains cluster consistency with Git but cannot build or validate application code, making Jenkins responsible for CI/CD automation. These layers were chosen not for popularity, but because each solves a capability gap the others cannot, creating clear operational boundaries and a production-aligned deployment workflow.

- Modular **Terraform over monolithic:** three independent modules (networking, database, EKS) with separate state keys per environment. A failure in EKS provisioning cannot trigger RDS rollback, and each module is independently testable before the next is touched.
- Remote **state with S3 + DynamoDB + KMS:** versioned S3 state enables rollback to any previous infrastructure snapshot; DynamoDB locking prevents concurrent apply corruption; KMS encryption ensures sensitive values never appear in plaintext in S3.
- Security **group layering by source reference:** ALB (application load balancer) accepts 80(http)/443 (https) from the internet; EKS nodes accept only from ALB SG (security group); RDS (database) accepts only port 5432 (Postgre SQL) from EKS node SG. No rule references a CIDR (range) where a SG reference works blast radius of any misconfiguration is contained to one boundary.
- OIDC (Protocol) **provider and EKS add-ons provisioned by Terraform, not manually:** OIDC is the prerequisite for IRSA and must exist before any ServiceAccount needs it. CoreDNS, kube-proxy, and VPC CNI (plugin) are pinned to specific versions in Terraform, preventing version drift between dev and staging.
- Bootstrap **bridge solves the Terraform-to-FluxCD handoff:** Terraform creates the cluster but cannot install FluxCD. An idempotent orchestration script runs post-apply: updates kubeconfig, bootstraps FluxCD, and forces initial sync. Safe to re-run at any point without side effects.
- Jenkins **deployed in-cluster via FluxCD HelmRelease:** Jenkins itself is GitOps-managed. Configuration changes go through Git, not the Jenkins UI. JCAsC (Jenkins Configuration as Code) handles credential injection and job seeding on first boot, eliminating manual setup entirely.
- **Stage 5 closes the GitOps loop:** the pipeline polls `kubectl get helmrelease` until `Ready=True` or a 5-minute timeout. On timeout, FluxCD events are retrieved and surfaced as the pipeline failure reason. A green build is a guarantee the application is running in the cluster not that an image was pushed.

One-Click Environment Provisioner — end-to-end architecture

Terraform → Bootstrap → FluxCD GitOps → Jenkins CI/CD → EKS · GitOps loop closes via infra-config repo



Decision Point	Chosen Tool	Alternative	Reason for Choice
IaC Tool	Terraform	Pulumi / AWS CDK	Mature AWS support, easier state management, simpler debugging
GitOps Engine	FluxCD	ArgoCD	Pull-based security model and native dependency ordering
Container Platform	Amazon EKS	ECS Fargate	Full Kubernetes ecosystem with HPA, IRSA, RBAC, and Cluster Autoscaler
Database	RDS PostgreSQL	Aurora PostgreSQL	Lower cost and sufficient for MVP implementation
Image Registry	Amazon ECR	Docker Hub	Native AWS IAM integration and no external pull limits
Jenkins Hosting	In-cluster Jenkins	External Jenkins Server	Fully GitOps-managed, reproducible, and automatically provisioned

Time constraint: Each layer is independently deployable and testable within its week, so if Terraform runs over, FluxCD work begins on a pre-existing cluster without blocking the team.

Compute and cost constraint: Dev uses t3.medium instances and single-AZ RDS to minimise AWS spend, with Infracost integrated into the pipeline to surface cost deltas before any apply reaches AWS.

State constraint: Remote S3 state with DynamoDB locking is provisioned first via the bootstrap module, not added later, because every subsequent layer depends on it being correct from day one.

MVP: Working Terraform modules, FluxCD managing platform components, Jenkins running all six stages.
Stretch goals: Drift detection, Infracost, Conftest policy gates, and environment promotion are sequenced, each independent of the others, so any can be dropped without affecting the core submission

3. Creativity & Novel Contributions: Beyond the minimum requirements, we propose five independent extensions.

#	Innovation	Gap Identified	Creativity	Why it matters
3.1	IRSA pod-level least privilege	Nothing about IAM scoping	A compromised pod inherits every permission on its node. We scope IAM roles per ServiceAccount via OIDC so each pod carries only what its function requires	Blast radius of any breach is contained to one service, not the entire AWS account
3.2	Drift detection proactive compliance	Nothing about post-provisioning	Infrastructure diverges silently through manual edits and partial failures. A nightly read-only <code>terraform plan</code> with exit-code-driven Slack alerting catches this before it becomes a deployment failure	Converts a crisis discovery moment into a routine compliance check
3.3	Multi-environment and multi availability zone as a validation strategy	Provision one environment and multi availability zone on cloud.	A module that works in one environment has hidden hardcoded assumptions. Provisioning dev, staging, and production from the same codebase surfaces those assumptions before they become incidents. Multi availability zone used because deployments are primarily used for high availability, durability, and fault tolerance.	Environment support reframed as a module correctness test, not a feature. Multi AZ eliminates SPOF (single point of failure)
3.4	Cost governance via Infracost blocking gate	Nothing about cost	A working provisioner can silently introduce significant cost increases invisible until the AWS bill arrives. Infracost is integrated as a pipeline gate if estimated monthly delta exceeds a threshold, the pipeline blocks before anything reaches AWS	Cost becomes a first-class deployment constraint, not an afterthought.
3.5	Closing the GitOps loop redefining green	Build and deploy a pipeline	Registry push and cluster deployment are two different events most pipelines treat as one. We poll FluxCD's HelmRelease status post-deploy and fail the pipeline if reconciliation does not confirm healthy within five minutes	A green build guarantees the application is running in the cluster not just that an image exists in a registry

4. Feasibility & Planning: 3-Week Execution Plan

Period	Milestone	Deliverable	Acceptance Criteria
Week 1			
Days 1–2	Bootstrap + Networking	S3 state bucket + DynamoDB lock table via bootstrap module. VPC /16, 3 public + 3 private subnets across 2 AZs, IGW, NAT GW, route tables, security groups for ALB, EKS nodes, and RDS	<code>terraform apply</code> succeeds clean; VPC visible in AWS console with correct CIDRs; DynamoDB lock prevents concurrent apply
Days 3–4	Database	RDS module: PostgreSQL 14, db.t3.micro, private subnet group, parameter group, Secrets Manager secret. RDS security group allows inbound only from EKS node security group	RDS endpoint reachable from bastion (primary access point for private network) in VPC; credentials retrieved from Secrets Manager only never from <code>terraform output</code>
Days 5–6	EKS + Remote State	EKS cluster, managed node group t3.medium ×2, OIDC provider, add-ons (CoreDNS, VPC CNI, kube-proxy). Remote state confirmed in S3	<code>kubectl get nodes</code> shows 2 Ready nodes; OIDC issuer URL matches cluster; second <code>terraform apply</code> produces zero changes
Day 7	Multi-Env + IRSA	Staging <code>.tfvars</code> file. IRSA roles for backend API, Jenkins agent, and Cluster Autoscaler each scoped independently	<code>terraform plan</code> for staging shows correct deltas; IRSA role ARNs annotated on ServiceAccounts; no hardcoded values anywhere in modules
Week 2			
Days 8–9	FluxCD Bootstrap	FluxCD installed via idempotent orchestration script. <code>gitops-infra</code> repo structured with <code>clusters/</code> , <code>infrastructure/</code> , <code>apps/</code> directories	<code>flux get kustomizations</code> shows Reconciled; git commit syncs to cluster within 60s; re-running bootstrap script produces zero changes
Days 10–11	Ingress + Secrets	cert-manager HelmRelease. ingress-nginx HelmRelease with <code>dependsOn: cert-manager</code> . External Secrets Operator with SecretStore pointing to AWS Secrets Manager via IRSA	HTTP traffic routes to placeholder service; External Secret Operator syncs RDS credentials to Kubernetes Secret; secret never appears in Terraform state or pipeline logs
Days 12–13	Monitoring + Alerting	kube-prometheus-stack HelmRelease (Prometheus + Grafana + Alertmanager). Slack webhook configured. Flux Notification Controller pointing to Slack	Grafana accessible via ingress; CPU/memory dashboard shows live node metrics; test alert fires to Slack; FluxCD sync failure triggers Slack notification
Day 14	Jenkins	Jenkins HelmRelease in <code>gitops-infra</code> . JCasC for credentials and job seed. ECR repository. IRSA role scoped to ECR push only	Jenkins UI accessible via ingress; test pipeline pushes image to ECR using IRSA no long-lived keys stored anywhere
Week 3			
Days 15–16	Application Pipeline	Jenkins 6-stage pipeline for frontend, backend, and db-migration. Stage 6 updates image tag in <code>gitops-infra</code> via Git commit and polls HelmRelease status	<code>git push</code> to app repo triggers full pipeline; new image running in EKS within minutes end-to-end; pipeline fails if HelmRelease not healthy within minutes
Days 17–18	Autoscaling + Load Test	HPA for frontend (CPU >60%) and backend (CPU >70%). Cluster Autoscaler with IRSA role. PodDisruptionBudgets configured. k6 load test script	k6 triggers HPA scale-up; Cluster Autoscaler provisions new node within minutes; zero pods stuck in Pending state during scale event
Day 19	Stretch + Demo	Drift detection nightly Jenkins job. Infracost integration. Full end-to-end demo from clean state. Runbook documentation complete	Demo runs clean end-to-end in under minutes; drift detection fires correctly on a manually introduced test change; Infracost posts cost delta on every plan

Risk	Likelihood	Impact	Fallback
AWS quota limits block EKS provisioning	Medium	High	Request quota increase Day 1; fall back to t3.small ×1
Terraform state corruption from interrupted apply	Low	High	S3 versioning for rollback; DynamoDB lock prevents concurrent runs
EKS bootstrap race condition nodes fail to register	Medium	High	<code>null_resource</code> depends_on enforces ordering; manual aws-auth patch documented
App codebase not provided or has complex build	Medium	High	Placeholder React + Node.js + PostgreSQL app pre-containerised and ready
EKS → RDS connectivity blocked by SG misconfiguration	Low	Medium	SG rules reference source SG not CIDRs; port-forward test validates in Week 1
FluxCD cannot access private Git repository	Low	Medium	SSH deploy key as Kubernetes Secret; fallback to public repo with branch protection
External Secrets Operator IRSA misconfiguration	Low	Medium	Manual Secrets Manager → Kubernetes Secret for MVP; ESO as Week 3 stretch

5. Team Composition & Readiness

Member	Role	Core Responsibilities	Key Ownership
Vaishnavi J P	Cloud Infrastructure & GitOps Lead	Terraform modules for networking, EKS, and RDS · Remote state bootstrap (S3 + DynamoDB) · OIDC provider and IRSA role configuration · Security group design and network isolation · FluxCD bootstrap and gitops-infra repository structure · All HelmRelease definitions for platform components · External Secrets Operator integration with AWS Secrets Manager	Everything from AWS resource lifecycle through to GitOps reconciliation the foundation every other component depends on
Vivek G	DevOps & Application Pipeline Lead	Jenkins 6-stage pipeline design and Jenkinsfile authoring · Multi-stage Dockerfiles for frontend, backend, and DB migration runner · Trivy vulnerability scanning integration · ECR image push via IRSA (no long-lived keys) · HelmRelease status polling in Stage 6 · HPA and Cluster Autoscaler configuration · k6 load testing · Prometheus and Grafana dashboard setup · Infracost integration	Everything from application build through to deployment verification, autoscaling, and observability

As of now, we have done this much for our prototype, this is just a starter to the project.

Link : <https://github.com/vaishjp/One-Click-Ops>

- Users interact through a web-based frontend platform** that allows registration, Kubernetes YAML uploads, and real-time deployment monitoring without manually using `kubectl` or cluster commands.
- A FastAPI backend automates Kubernetes namespace provisioning** for each user by creating isolated namespaces with resource quotas, default limits, and network policies to ensure secure multi-tenant deployments inside a shared AWS EKS cluster.
- Uploaded manifests are committed to a GitHub repository monitored by FluxCD**, which continuously synchronizes changes to the EKS cluster using GitOps, while the frontend displays live deployment, pod health, and service status updates for a one-click deployment experience.

Technical Stack Summary:

Infrastructure: Terraform, AWS Provider, S3 + DynamoDB remote state, KMS.

Compute: EKS, EC2 managed nodes, Cluster Autoscaler.

Database: RDS PostgreSQL, Secrets Manager
GitOps: FluxCD, Helm, Kustomize, HelmRelease CRDs.

Platform: ingress-nginx, cert-manager, kube-prometheus-stack, Flagger.

CI/CD: Jenkins, ECR, Docker multistage builds, Trivy, Infracost, k6.